

C++ - Interfaces Qt

Patricio Olivares

Ingeniería Civil Telemática
Departamento de Electrónica
Universidad Técnica Federico Santa María

¿Qué es Qt?

- Qt (pronunciado çute) es un **framework** de desarrollo de aplicaciones que incluye múltiples bibliotecas
- Fue inicialmente desarrollado para el lenguaje C++, pero en la actualidad es posible utilizarlo en otros lenguajes (Ej: PyQt/PySide para Python)
- Provee mejoras a las bibliotecas nativas de C++ y permite códigos multiplataforma.
- Algunas plataformas compatibles: Windows, Linux/Unix, MacOS, iOS, Android
- Puede encontrar la última versión de Qt en el siguiente enlace: <https://www.qt.io/>.
- La última versión de Qt es Qt6.

Módulos Qt

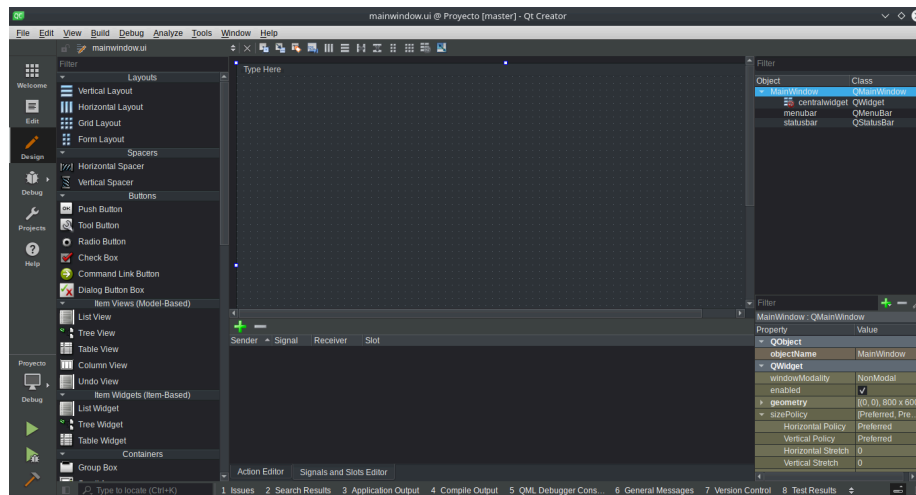
Existen dos tipos de módulos en Qt:

- **Módulos Core-Essential:** Son parte de las bibliotecas estándar de Qt. Ejemplo: *QtCore*, *QtGui*, *QtWidgets*, etc.
- **Módulos add-ons (agregados):** Son módulos externos al desarrollo de Qt (desarrollados por terceros). Ejemplos: *Qt3D*, *QtBluetooth*, *QtSensors*, etc.

Qt Creator

- Qt, al ser un conjunto de bibliotecas, pueden ser integradas en cualquier IDE que lo permita. Incluso, es posible compilar programas Qt sin IDE.
- Qt Creator es un IDE multiplataforma para desarrollo de programas en C++ creado por el equipo de desarrollo Qt. Está orientado principalmente gráficas fácilmente utilizando las bibliotecas de desarrollo que provee Qt.

Qt Creator

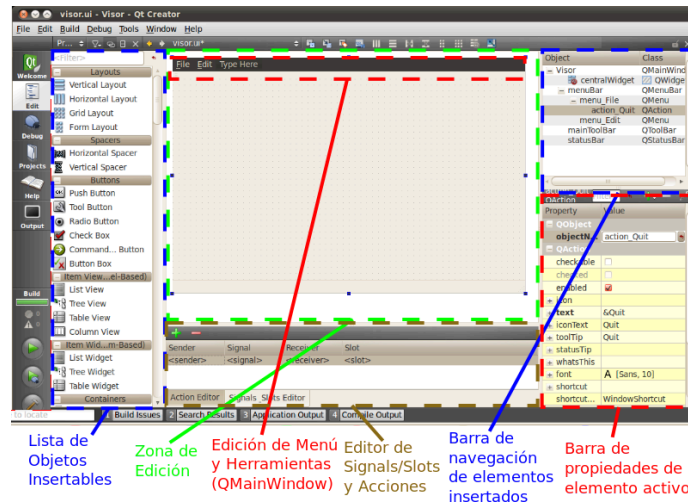


Qt Creator

Qt Creator está compuesto por:

- **Editor de Texto:** Este posee varias funcionalidades, como autocompletado, revisión de sintaxis, indentación automática, etc.
- **Qt Designer:** Para creación de interfaces gráficas sencillas
- Otras funcionalidades (Ej: Debug, conexión con sistemas de versionamiento (GIT, SVN, otros, etc.))

Qt Creator

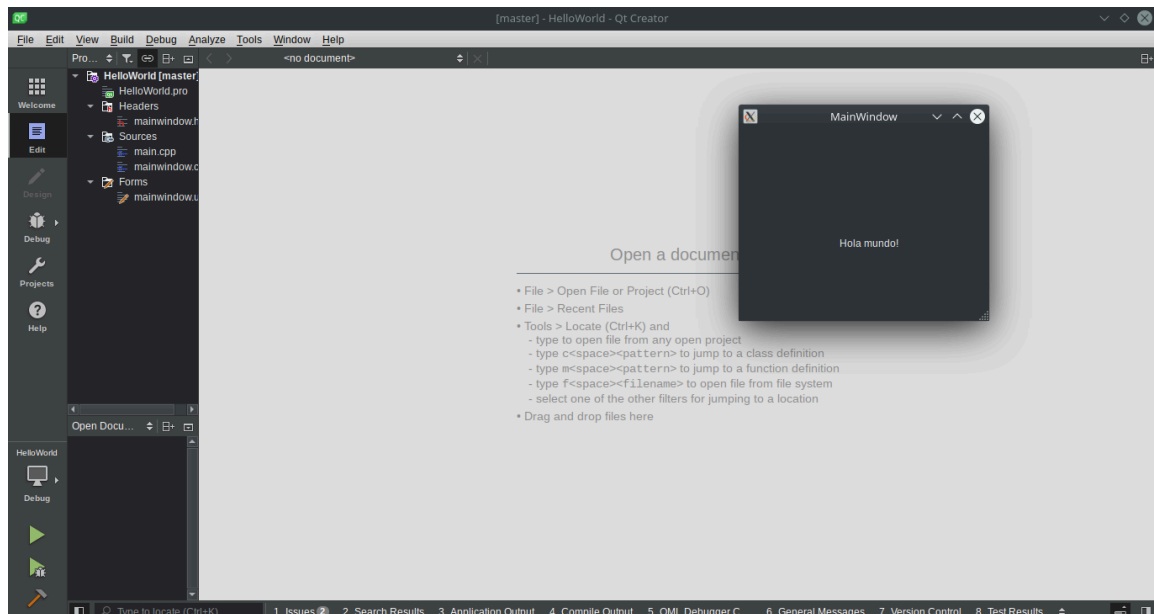


Qt Proyectos

Un proyecto Qt está compuesto por los siguientes elementos:

- **Headers:** Conjunto de archivos con extensión **.h**. Contiene las definiciones de las clases utilizadas en la aplicación.
- **Sources:** Conjunto de archivos con extensión **.cpp**. Contiene las implementaciones de los métodos de clase y funciones utilizadas.
- **Forms:** Conjunto de archivos con extensión **.ui**. Son archivos con formato **XML** que contienen las características y disposición de los distintos *elementos gráficos* de cada pantalla del programa. Estos archivos pueden ser modificados utilizando el *Qt Designer* de forma gráfica o modificando directamente los archivo **.ui**.
- **Archivo .pro:** Es el archivo que contiene las directivas de compilación para el comando **qmake**, comando encargado de generar *makefiles* para desarrollos Qt.

Qt - Mi primer proyecto: HelloWorld



Qt - HelloWorld: HelloWorld.pro

```

QT      += core gui #Libraries
greaterThan(QT_MAJOR_VERSION, 4): QT += widgets #Libraries
CONFIG += c++11
# The following define makes your compiler emit warnings if you use
# any Qt feature that has been marked deprecated (the exact
warnings
# depend on your compiler). Please consult the documentation of the
# deprecated API in order to know how to port your code away from
it.
DEFINES += QT_DEPRECATED_WARNINGS
# You can also make your code fail to compile if it uses
deprecated APIs.
# In order to do so, uncomment the following line.
# You can also select to disable deprecated APIs only up to a
certain version of Qt.
DEFINES += QT_DISABLE_DEPRECATED_BEFORE=0x060000    # disables all
the APIs deprecated before Qt 6.0.0
SOURCES += \
    main.cpp \
    mainwindow.cpp
HEADERS += \
    mainwindow.h
FORMS += \
    mainwindow.ui
# Default rules for deployment.
qnx: target.path = /tmp/${TARGET}/bin
else: unix:!android: target.path = /opt/${TARGET}/bin
!isEmpty(target.path): INSTALLS += target

```

Qt - HelloWorld: Headers/mainwindow.h

```
In [ ]: #ifndef MAINWINDOW_H
        #define MAINWINDOW_H
        #include <QMainWindow>
        // Se agrega la clase MainWindow actual al namespace Ui
        QT_BEGIN_NAMESPACE
        namespace Ui { class MainWindow; }
        QT_END_NAMESPACE
        // Clase que contiene definiciones de la ventana principal
        // Hereda de QMainWindow
        class MainWindow : public QMainWindow{
            // Macro que convierte la clase en un QObject
            Q_OBJECT
        public:
            // Constructor de ventana principal
            // Una ventana, puede estar asociada a otra ventana/dialog/widget. Si ex
            // esa asociación, *parent != nullptr
            MainWindow(QWidget *parent = nullptr);
            ~MainWindow();
        private:
            // Puntero que se hará apuntar a la interfaz gráfica definida en mainwir
            // Definición de esta clase se encuentra en el archivo "ui_mainwindow.h"
            // (generado automáticamente)
            Ui::MainWindow *ui;
        };
        #endif // MAINWINDOW_H
```

Qt - HelloWorld: Sources/main.cpp

```
In [ ]: #include "mainwindow.h"

#include <QApplication>

int main(int argc, char *argv[])
{
    // Se crea una instancia de aplicación
    QApplication a(argc, argv);
    // Dentro de la aplicación, se crea una instancia de MainWindow
    MainWindow w;
    // Se muestra la ventana principal
    w.show();
    // Se ejecuta la aplicación gráfica
    return a.exec();
}
```

Qt - HelloWorld: Sources/mainwindow.cpp

```
In [ ]: #include "mainwindow.h"
#include "ui_mainwindow.h"

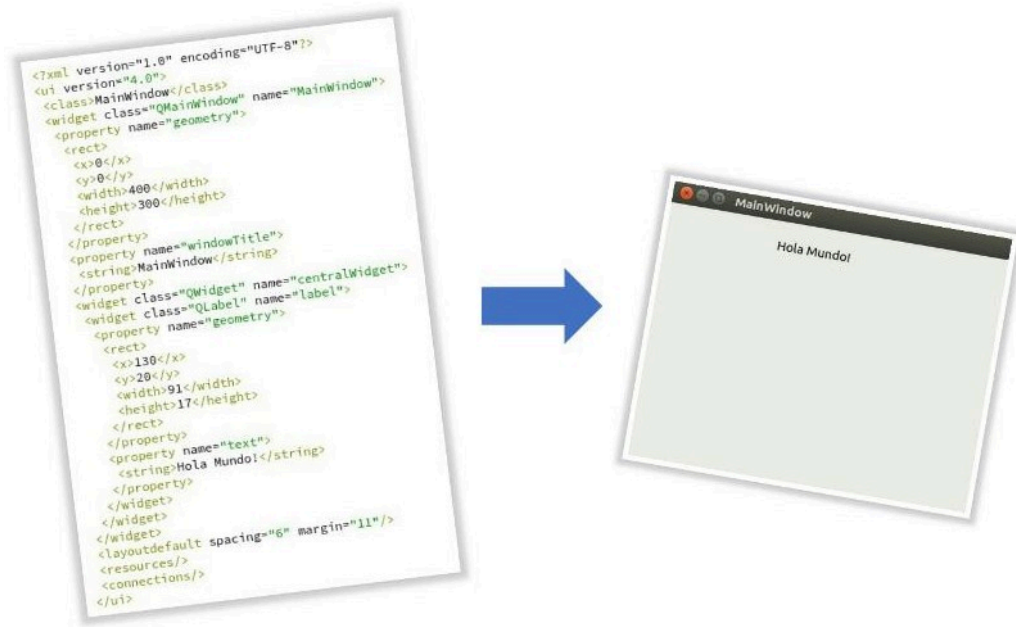
// Constructor de ventana principal. Se inicializa con el constructor de
// QMainWindow y su atributo ui con lista de inicialización
MainWindow::MainWindow(QWidget *parent)
    : QMainWindow(parent)
    , ui(new Ui::MainWindow)
{
    // Agrega a MainWindow todos los elementos que se configuran a través de
    ui->setupUi(this);
}

// Destructor de MainWindow. Libera puntero ui una vez se cierra la ventana
MainWindow::~MainWindow()
{
    delete ui;
}
```

Qt - HelloWorld: Forms/mainwindow.ui

```
<?xml version="1.0" encoding="UTF-8"?>
<ui version="4.0">
  <class>MainWindow</class>
  <widget class="QMainWindow" name="MainWindow">
    <property name="geometry">
      <rect>
        <x>0</x>
        <y>0</y>
        <width>800</width>
        <height>600</height>
      </rect>
    </property>
    <property name="windowTitle">
      <string>MainWindow</string>
    </property>
    <widget class="QWidget" name="centralwidget">
      <widget class="QLabel" name="label">
        <property name="geometry">
          <rect>
            <x>120</x>
            <y>120</y>
            <width>171</width>
            <height>31</height>
          </rect>
        </property>
      </widget>
    </widget>
  </widget>
</ui>
```

Qt - HelloWorld: Forms/mainwindow.ui



Qt - Qt Charts

- El módulo Qt Charts provee un conjunto de componentes para gráficos fáciles de usar.
- Usa el Qt Graphics View Framework (<https://doc.qt.io/qt-6/graphicsview.html>) que permite integrar fácilmente gráficos a las interfaces gráficas
- Permite la creación de gráficos de líneas, barras, tortas, polares, áreas, entre otros.
- Debe estar instalado previamente junto con las bibliotecas de Qt para poder utilizarlo.

Qt - Qt Charts: Configuración

- Incluir en el archivo **.pro** de tu proyecto Qt

QT += charts

- Incluir en el código la siguiente cabecera

```
#include <QtCharts>
```

En versiones anteriores a la versión 6 de Qt, también es necesario agregar lo siguiente

```
using namespace QtCharts;
```

- Para grabar imágenes en formato estándar, agregar en su código:

```
chartView->chart()->setAnimationOptions(QChart::NoAnimation);
QPixmap p = chartView->grab();
```

```
p.save("a.png", "PNG");
```

Qt - Qt Charts - QLineSeries (Gráficos de línea)

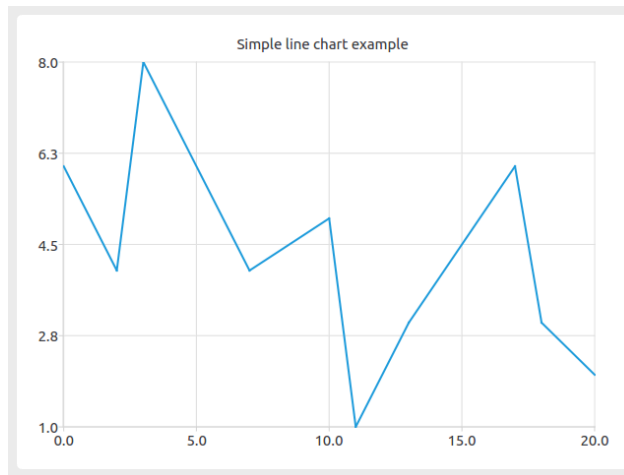
```
In [ ]: #include "mainwindow.h"
#include <QApplication>

int main(int argc, char *argv[])
{
    QApplication a(argc, argv);
    // Crear serie
    QLineSeries *series = new QLineSeries();
    series->append(0, 6);
    series->append(2, 4);
    series->append(3, 8);
    series->append(7, 4);
    series->append(10, 5);
    *series << QPointF(11, 1) << QPointF(13, 3)
    << QPointF(17, 6) << QPointF(18, 3)
    << QPointF(20, 2);
```

Qt - Qt Charts - QLineSeries (Gráficos de línea)

```
In [ ]: // Crear gráfico e insertar serie
QChart *chart = new QChart();
chart->legend()->hide();
chart->addSeries(series);
chart->createDefaultAxes();
chart->setTitle("Simple line chart example");
// Crear widget QChartView (e insertarlo en una ventana):
QChartView *chartView = new QChartView(chart);
chartView->setRenderHint(QPainter::Antialiasing);
MainWindow window;
window.setCentralWidget(chartView);
window.show();
return a.exec();
}
```

Qt - Qt Charts - QLineSeries (Gráficos de línea)



Qt - Qt Charts - QBarSeries (Gráficos de barras)

```
In [ ]: #include "mainwindow.h"
#include <QApplication>

int main(int argc, char *argv[])
{
    QApplication a(argc, argv);
    // Crear conjuntos
    QBarSet *set0 = new QBarSet("Jane");
    QBarSet *set1 = new QBarSet("John");
    QBarSet *set2 = new QBarSet("Axel");
    QBarSet *set3 = new QBarSet("Mary");
    QBarSet *set4 = new QBarSet("Samantha");
    *set0 << 1 << 2 << 3 << 4 << 5 << 6;
    *set1 << 5 << 0 << 0 << 4 << 0 << 7;
    *set2 << 3 << 5 << 8 << 13 << 8 << 5;
    *set3 << 5 << 6 << 7 << 3 << 4 << 5;
    *set4 << 9 << 7 << 5 << 3 << 1 << 2;
    // Crear series
    QBarSeries *series = new QBarSeries;
    series->append(set0);
    series->append(set1);
    series->append(set2);
    series->append(set3);
    series->append(set4);
}
```

Qt - Qt Charts - QBarSeries (Gráficos de barras)

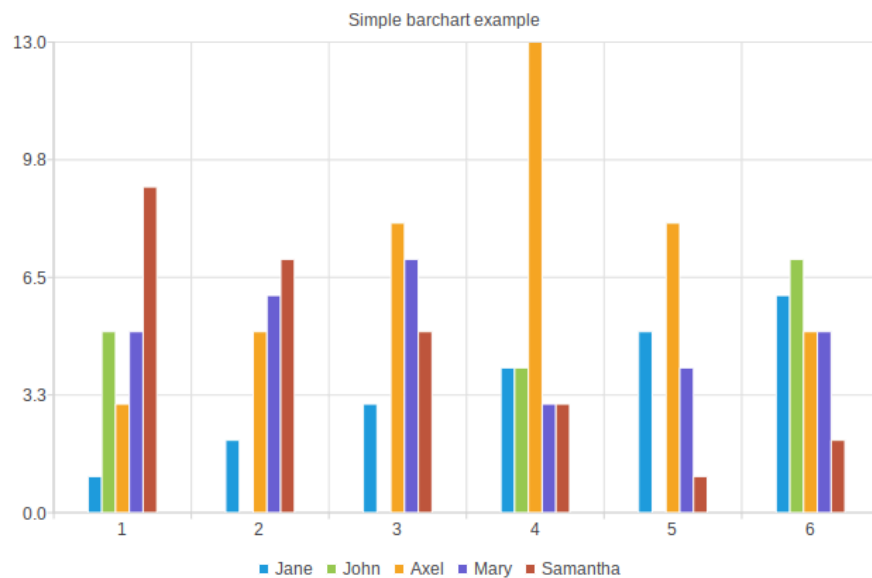
```
In [ ]: // Crear gráfico e insertar serie
QChart *chart = new QChart();
chart->legend()->setVisible(true);
chart->legend()->setAlignment(Qt::AlignBottom);
```

```

chart->addSeries(series);
chart->createDefaultAxes();
chart->setTitle("Simple line chart example");
// Crear widget QChartView (e insertarlo en una ventana):
QChartView *chartView = new QChartView(chart);
chartView->setRenderHint(QPainter::Antialiasing);
MainWindow window;
window.setCentralWidget(chartView);
window.show();
return a.exec();
}

```

Qt - Qt Charts - QBarSeries (Gráficos de barras)



Qt - Qt Charts - QHorizontalBarSeries (Gráficos de barras horizontal)

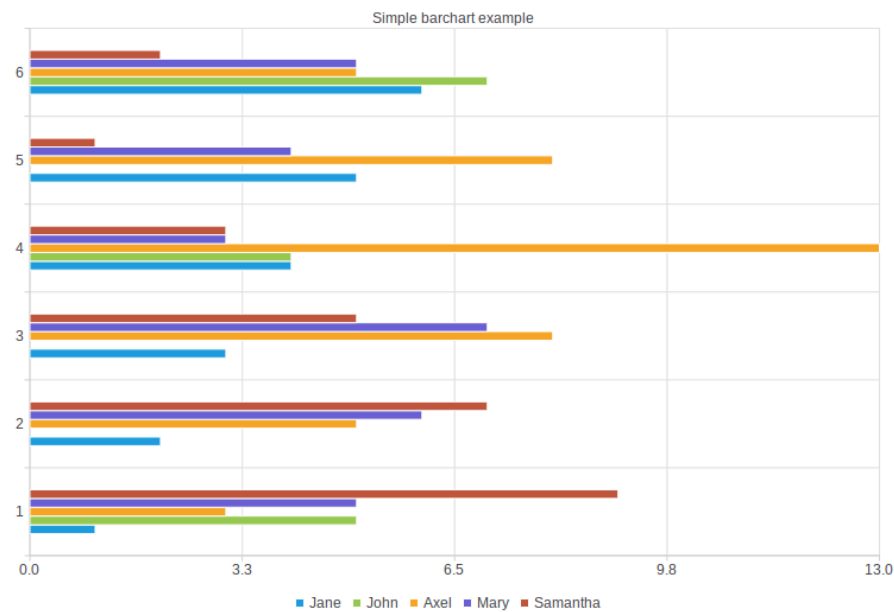
- Gráfico de barra horizontal es prácticamente idéntico a gráfico de barras.
- Series horizontales

```

QHorizontalBarSeries *series = new QHorizontalBarSeries();
series->append(set0);
series->append(set1);
series->append(set2);
series->append(set3);
series->append(set4);

```

Qt - Qt Charts - QHorizontalBarSeries (Gráficos de barras horizontal)



Qt - Qt Charts - QPieSeries (Gráficos de torta)

```
In [ ]: #include "mainwindow.h"
#include <QApplication>

int main(int argc, char *argv[])
{
    QApplication a(argc, argv);

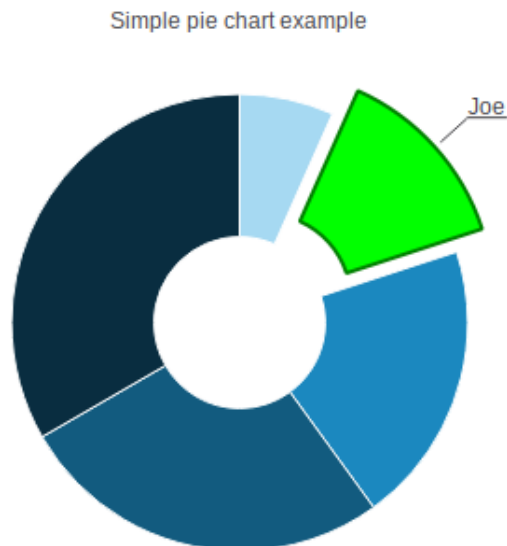
    // Gráfico de torta. Serie de torta
    QPieSeries *series = new QPieSeries();
    series->append("Jane", 1);
    series->append("Joe", 2);
    series->append("Andy", 3);
    series->append("Barbara", 4);
    series->append("Axel", 5);
    series->setPieSize(0.45);
    // Acomodar series como tortas o donas concéntricas
    series->setHoleSize(0.3);
    series->setPieSize(0.8);
    // Editar corte del gráfico
    QPieSlice *slice = series->slices().at(1);
    slice->setExploded();
    slice->setLabelVisible();
}
```

```
slice->setPen(QPen(Qt::darkGreen, 2));  
slice->setBrush(Qt::green);
```

Qt - Qt Charts - QPieSeries (Gráficos de torta)

```
In [ ]: // Crear gráfico e inserir serie  
QChart *chart = new QChart();  
chart->legend()->hide();  
chart->addSeries(series);  
chart->createDefaultAxes();  
chart->setTitle("Simple pie chart example");  
  
QChartView *chartView = new QChartView(chart);  
chartView->setRenderHint(QPainter::Antialiasing);  
  
MainWindow window;  
window.setCentralWidget(chartView);  
window.show();  
return a.exec();  
}
```

Qt - Qt Charts - QPieSeries (Gráficos de torta)



Qt - Qt Charts - QAreaSeries (Gráficos de área)

```
In [ ]: #include "mainwindow.h"
#include <QApplication>

int main(int argc, char *argv[])
{
    QApplication a(argc, argv);

    // Se utilizan dos QLineSeries que definen los límites superior e inferior
    QLineSeries *series0 = new QLineSeries();
    QLineSeries *series1 = new QLineSeries();
    *series0    << QPointF(1, 5) << QPointF(3, 7) << QPointF(7, 6) << QPointF(12, 6) << QPointF(16, 7) << QPointF(18, 5);
    *series1    << QPointF(1, 3) << QPointF(3, 4) << QPointF(7, 3) << QPointF(12, 3) << QPointF(16, 4) << QPointF(18, 3);
    QAreaSeries *series = new QAreaSeries(series0, series1);
```

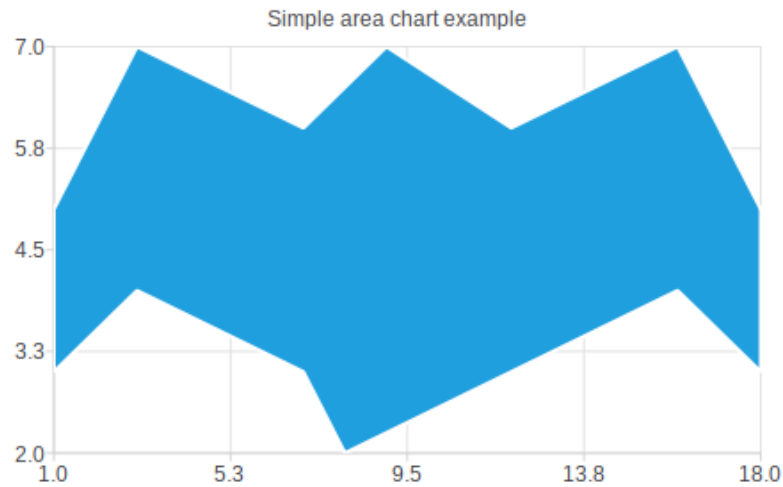
Qt - Qt Charts - QAreaSeries (Gráficos de área)

```
In [ ]: // Crear gráfico e insertar serie
QChart *chart = new QChart();
chart->legend()->hide();
chart->addSeries(series);
chart->createDefaultAxes();
chart->setTitle("Simple area chart example");

QChartView *chartView = new QChartView(chart);
chartView->setRenderHint(QPainter::Antialiasing);

MainWindow window;
window.setCentralWidget(chartView);
window.show();
return a.exec();
}
```

Qt - Qt Charts - QAreaSeries (Gráficos de área)



Qt - Qt Charts - QPolarChart (Gráfico polar)

- Para crear un gráfico polar, se debe utilizar la clase QPolarChart en vez de QChart

QPolarChart *chart = new QPolarChart();

```
In [ ]: #include "mainwindow.h"
#include <QApplication>

int main(int argc, char *argv[])
{
    QApplication a(argc, argv);
    // Variables que almacenan los ángulos mínimo y máximo del gráfico
    const qreal angularMin = -100;
    const qreal angularMax = 100;
    // Variables que almacenan los valores de radio mínimos y máximo
    const qreal radialMin = -100;
    const qreal radialMax = 100;
```

Qt - Qt Charts - QPolarChart (Gráfico polar)

```
In [ ]: // Creación de serie de datos para graficar
QScatterSeries *series1 = new QScatterSeries();
series1->setName("scatter");
for (int i = angularMin; i <= angularMax; i += 10)
    series1->append(i, (i / radialMax) * radialMax + 8.0);
QPolarChart *chart = new QPolarChart();
chart->addSeries(series1);
// Creación de eje angular
QValueAxis *angularAxis = new QValueAxis();
```

```
// Configuración visual de eje angular
angularAxis->setTickCount(9);
angularAxis->setLabelFormat("%.1f");
angularAxis->setShadesVisible(true);
angularAxis->setShadesBrush(QBrush(QColor(249, 249, 255)));
// Conexión de eje angular con gráfico
chart->addAxis(angularAxis, QPolarChart::PolarOrientationAngular);
```

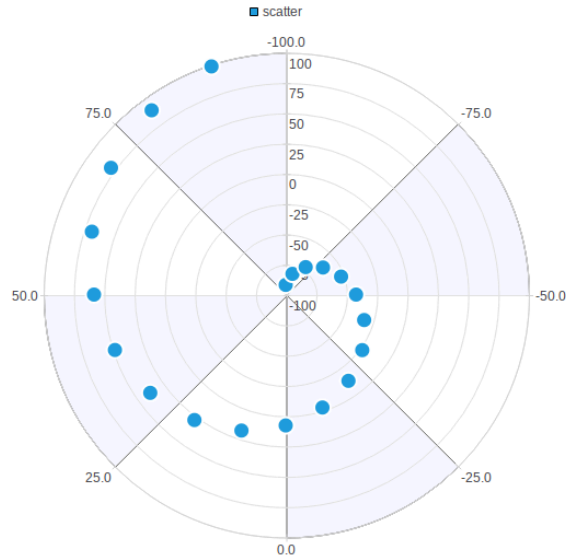
Qt - Qt Charts - QPolarChart (Gráfico polar)

```
In [ ]: // Creación de eje radial
QValueAxis *radialAxis = new QValueAxis();
//Configuración de eje radial
radialAxis->setTickCount(9);
radialAxis->setLabelFormat("%d");
// Conexión de eje radial con gráfico
chart->addAxis(radialAxis, QPolarChart::PolarOrientationRadial);
// Conexión de ejes con serie
series1->attachAxis(radialAxis);
series1->attachAxis(angularAxis);
// Rango de ejes
radialAxis->setRange(radialMin, radialMax);
angularAxis->setRange(angularMin, angularMax);

QChartView *chartView = new QChartView();
chartView->setChart(chart);
chartView->setRenderHint(QPainter::Antialiasing);

MainWindow window;
window.setCentralWidget(chartView);
window.resize(800, 600);
window.show();
return a.exec();
}
```

Qt - Qt Charts - QPolarChart (Gráfico polar)



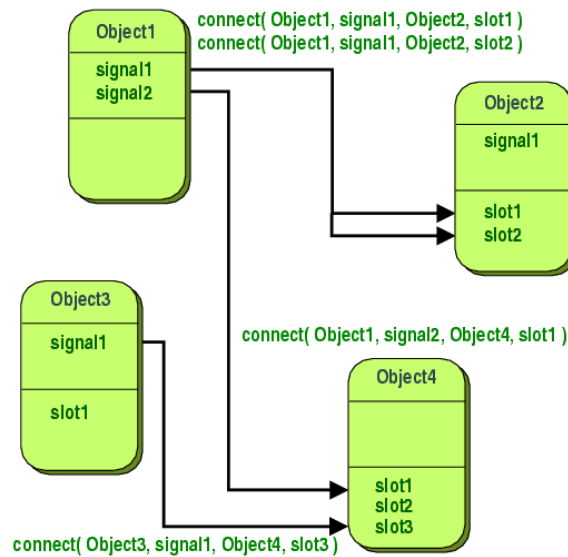
Qt - Signals y slots

- Permiten la comunicación entre objetos de tipo QObject.
- Una señal (**signal**) es emitida por un objeto para señalar la ocurrencia de un evento o cambio de interés particular.
- Una ranura (**slot**) es un método que es llamado en respuesta a una señal particular.
- Una clase puede tener tanto signals como slots

Qt - Signals y slots

- **Signal**
 - Son funciones/métodos de acceso público que **no deben ser implementados y no retornan valores**.
 - Señales son emitidas por un objeto para señalar algún evento de interés (definido por el programador).
- **Slot**
 - Son funciones/métodos que pueden ser llamados normalmente. Su única diferencia es que se pueden conectar con signals.
 - Un slot conectado a una signal es llamado cuando la señal o señales a las que se encuentra conectado son emitidas.

Qt - Signals y slots



Qt - Signals y slots - Ejemplo

```
In [ ]: // Counter.h
#include <QObject>

class Counter: public QObject{
    Q_OBJECT

public:
    Counter(){m_value=0;}
    int value(){return m_value;}

public slots:
    void setValue(int value);

signals:
    void valueChanged(int newValue);

private:
    int m_value;

};
```

Qt - Signals y slots - Ejemplo

```
In [ ]: // Counter.cpp
#include "Counter.h"

void Counter::setValue(int value){
    if(value != m_value){
        m_value = value;
        emit valueChanged(value);
    }
}
```

```
}  
}
```

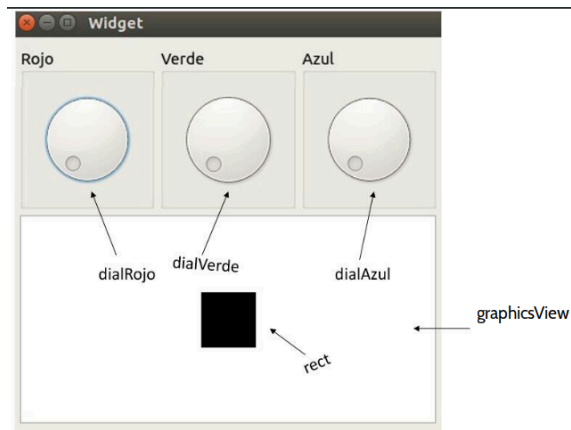
Qt - Signals y slots - Ejemplo

```
In [ ]: // main.cpp  
#include <QCoreApplication>  
#include "Counter.h"  
#include <iostream>  
  
int main(int argc, char *argv[])  
{  
    QCoreApplication a(argc, argv);  
  
    Counter *p = new Counter();  
    Counter *q = new Counter();  
    QObject::connect(p, SIGNAL(valueChanged(int)), q, SLOT(setValue(int)));  
  
    p->setValue(12);  
    std::cout << "El valor de p es " << p->value() << " y el valor de q es "  
  
    q->setValue(48);  
    std::cout << "El valor de p es " << p->value() << " y el valor de q es "  
  
    /*  
        El valor de p es 12 y el valor de q es 12  
        El valor de p es 12 y el valor de q es 48  
    */  
    return a.exec();  
}
```

Qt - Ejemplo: Dial Color

- Se desea desarrollar un programa que cuenta con 3 diales y un cuadrado de dimensiones 50px por 50px y de color inicialmente negro según aparece en la imagen.
- Cada uno de los diales representa los colores rojo, verde y azul respectivamente y cuyos valores van del 0 al 255. El color del cuadrado viene dado por la combinación de los valores de la posición de cada uno de los diales en codificación RGB. Si uno de los diales se mueve, el color del cuadrado se actualiza con los nuevos valores presentes.

Qt - Ejemplo: Dial Color



Qt - Ejemplo: Dial Color

```
In [ ]: // Sources/main.cpp
#include "widget.h"
#include <QApplication>

int main(int argc, char *argv[])
{
    QApplication a(argc, argv);
    Widget w;
    w.show();

    return a.exec();
}
```

Qt - Ejemplo: Dial Color

```
In [ ]: // Headers/rectanglecolor.h

class RectangleColor:public QObject,public QGraphicsRectItem{
    Q_OBJECT
public:
    RectangleColor(int x, int y, int w, int h);

signals:

public slots:
    void changeRed(int);
    void changeGreen(int);
    void changeBlue(int);

private:
    int rojo = 0;
    int verde = 0;
    int azul = 0;
    void updateColor();
};
```

```
#endif // RECTANGLECOLOR_H
```

Qt - Ejemplo: Dial Color

```
In [ ]: // Sources/rectanglecolor.cpp
#include "rectanglecolor.h"

RectangleColor::RectangleColor(int x, int y, int w, int h) :
    QGraphicsRectItem(x,y,w,h)
{
    updateColor();
}

void RectangleColor::changeRed(int rojo){
    this->rojo = rojo;
    updateColor();
}

void RectangleColor::changeGreen(int verde){
    this->verde = verde;
    updateColor();
}

void RectangleColor::changeBlue(int azul){
    this->azul = azul;
    updateColor();
}

void RectangleColor::updateColor(){
    QColor color(this->rojo, this->verde, this->azul);
    this->setBrush(QBrush(color));
}
```

Qt - Ejemplo: Dial Color

```
In [ ]: // Headers/widget.h
#ifndef WIDGET_H
#define WIDGET_H

#include <QWidget>
#include <QGraphicsScene>
#include "rectanglecolor.h"

namespace Ui {
class Widget;
}

class Widget : public QWidget
{
    Q_OBJECT
```

```

public:
    explicit Widget(QWidget *parent = 0);
    ~Widget();

private:
    Ui::Widget *ui;
    QGraphicsScene *scene;
    RectangleColor *rect;
};

#endif // WIDGET_H

```

Qt - Ejemplo: Dial Color

```

In [ ]: // Sources/widget.cpp
#include "widget.h"
#include "ui_widget.h"

Widget::Widget(QWidget *parent) :
    QWidget(parent),
    ui(new Ui::Widget)
{
    ui->setupUi(this);
    scene = new QGraphicsScene(this);
    ui->graphicsView->setScene(scene);
    rect = new RectangleColor(0, 0, 50, 50);
    scene->addItem(rect);
    connect(ui->dialRojo, SIGNAL(valueChanged(int)), rect, SLOT(changeRed(int)));
    connect(ui->dialVerde, SIGNAL(valueChanged(int)), rect, SLOT(changeGreen(int)));
    connect(ui->dialAzul, SIGNAL(valueChanged(int)), rect, SLOT(changeBlue(int)));
}

Widget::~~Widget()
{
    delete ui;
    delete scene;
    delete rect;
}

```